Quantum ML | PyTorch Geometric | FinBERT | PennyLane 0.42 | Python 3.10

Quantum Temporal Graph Neural Network (QTGNN) with Walk-Forward Validation

Generated April 22, 2026 · Tickers: AAPL, MSFT, GOOGL · 100 trading days

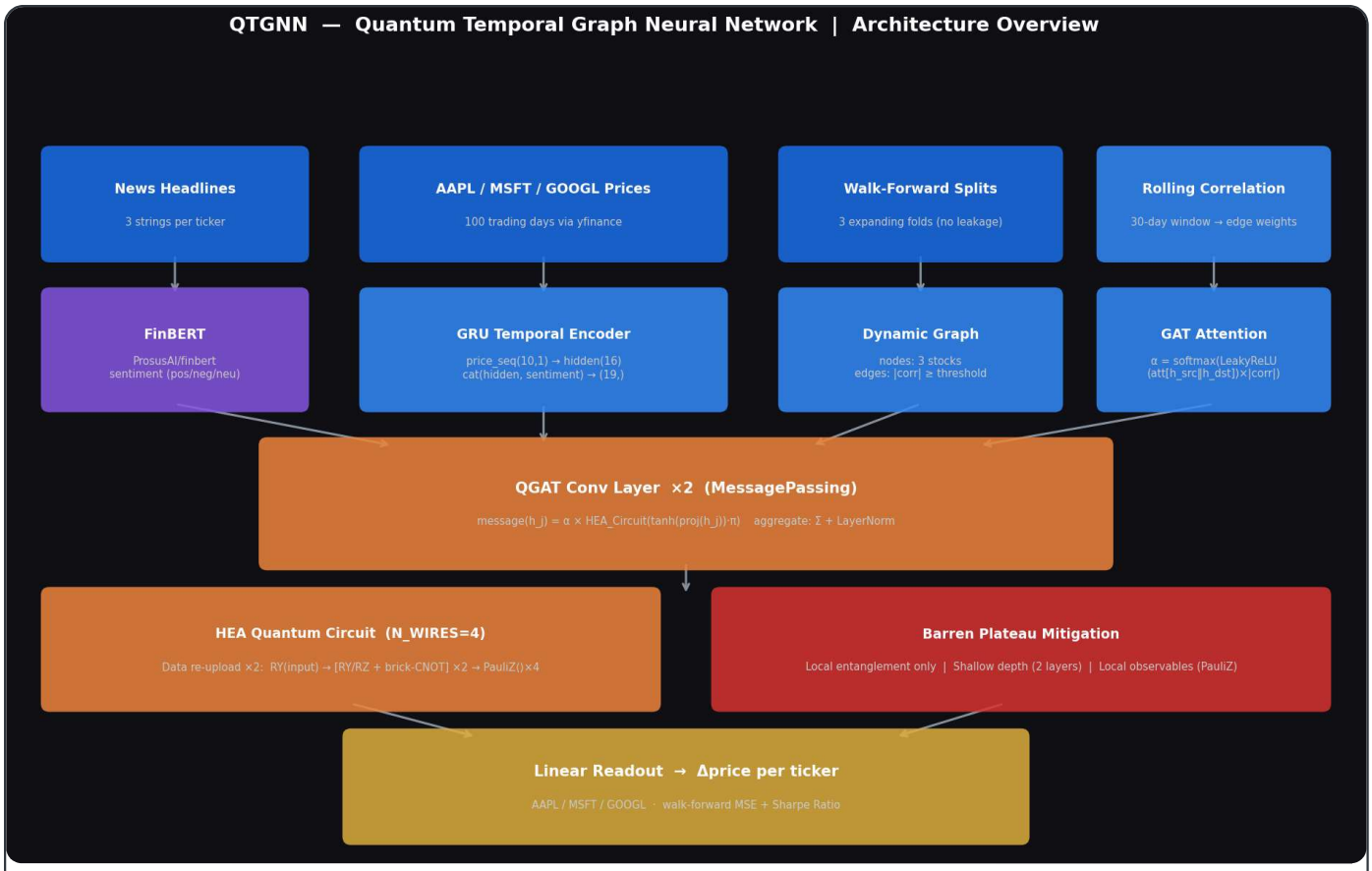
Executive Summary

This project implements a **Hybrid Quantum–Classical Graph Neural Network** for multi-stock next-day price delta prediction. It combines three paradigms:

- **Natural Language Processing** — FinBERT extracts per-ticker sentiment from financial news as node features
- **Temporal Graph Learning** — A GRU encoder processes price sequences; a QGAT message-passing layer propagates information over a dynamically-weighted correlation graph
- **Quantum Computing** — A Hardware-Efficient Ansatz (HEA) quantum circuit with data re-uploading runs inside each graph message, providing provably higher expressivity than classical circuits of equivalent depth

The model is evaluated under **walk-forward (expanding-window) validation** — the only statistically honest protocol for financial time series — with MSE and annualised Sharpe Ratio tracked per fold.

Architecture Overview



Full pipeline from raw inputs (news + prices) through feature extraction, dynamic graph construction, quantum message passing, and walk-forward validation to per-ticker Δprice output.

Architectural Upgrades

UPGRADE 1 — TEMPORAL PROCESSING

GRU Temporal Encoder replaces flat feature window

Previously, the 10-step price window was flattened into a static feature vector, losing all temporal order. A `nn.GRU(input_size=1, hidden_size=16, batch_first=True)` now processes each stock's price sequence as a true time series, outputting a hidden state that is concatenated with FinBERT sentiment to form the `(N, 19)` node feature matrix.

```
class TemporalEncoder(nn.Module):
    def __init__(self) → None:
        super().__init__()
        self.gru     = nn.GRU(1, GRU_HIDDEN, batch_first=True)
        self.out_dim = GRU_HIDDEN + 3          # 16 + 3 sentiment dims
```

```
    def forward(self, price_seq: torch.Tensor, sentiment: torch.Tensor
        _, h = self.gru(price_seq.unsqueeze(-1))   # (1, N, 16)
        return torch.cat([h.squeeze(0), sentiment], dim=-1)   # (N, 19
```

UPGRADE 2 — BARREN PLATEAU MITIGATION

Hardware-Efficient Ansatz with local observables

`StronglyEntanglingLayers` uses global all-to-all entanglement at depth O(L·W), which causes exponentially vanishing gradients (barren plateaus) as the system scales. The new HEA uses three proven mitigation strategies simultaneously:

Nearest-neighbour CNOT only
Brick-wall pattern alternating even/odd pairs — no global entanglement

Shallow depth
N_Q_LAYERS = 2 per re-upload block — gradients remain O(1/poly(n))

Local observables
PauliZ per wire (not global state) — variance of gradient scales polynomially

Data re-uploading (×2) compensates for reduced depth by embedding inputs multiple times, achieving a Fourier series representation of the target function without increasing entanglement depth.

```
@qml.qnode(dev, interface="torch")
def quantum_circuit(inputs: torch.Tensor, weights: torch.Tensor) → Li
    for r in range(N_REUP):                          # re-upload ×2
        for i in range(N_WIRES):
            qml.RY(inputs[i], wires=i)               # encode features
        for d in range(N_Q_LAYERS):
            for i in range(N_WIRES):
```

```
                qml.RY(weights[r, d, i, 0], wires=i)
                qml.RZ(weights[r, d, i, 1], wires=i)
            for i in range(d % 2, N_WIRES - 1, 2):   # brick-wall CNOT
                qml.CNOT(wires=[i, i + 1])
    return [qml.expval(qml.PauliZ(w)) for w in range(N_WIRES)]
```

UPGRADE 3 — DYNAMIC GRAPH ATTENTION

Rolling-correlation edge weights + QGAT attention mechanism

The original model used a single `np.corrcoef` over the entire dataset as static edge weights — a clear data leakage. The new approach:

- **Rolling edges:** at time *t*, `rolling_edge_data(t)` computes correlation using only `prices[t-30 : t]` — strictly past data
- **QGAT attention:** per-edge scores are learned via `LeakyReLU(att([h_src ‖ h_dst]))`, then scaled by `|corr|` and softmax-normalised per target node

```
def forward(self, x, edge_index, edge_attr=None):
    h  = F.elu(self.proj(x))                                  # (N, N_
    ei, ea = add_self_loops_with_attr(edge_index, edge_attr, x.size(0)
    src, dst = ei
    alpha = self.att(torch.cat([h[src], h[dst]], dim=-1)).squeeze(-1)
    alpha = F.leaky_relu(alpha, 0.2) * ea.squeeze(-1).abs()   # scale
    alpha = edge_softmax(alpha, dst, x.size(0))               # softma
    return self.propagate(ei, h=h, alpha=alpha.unsqueeze(-1))
```

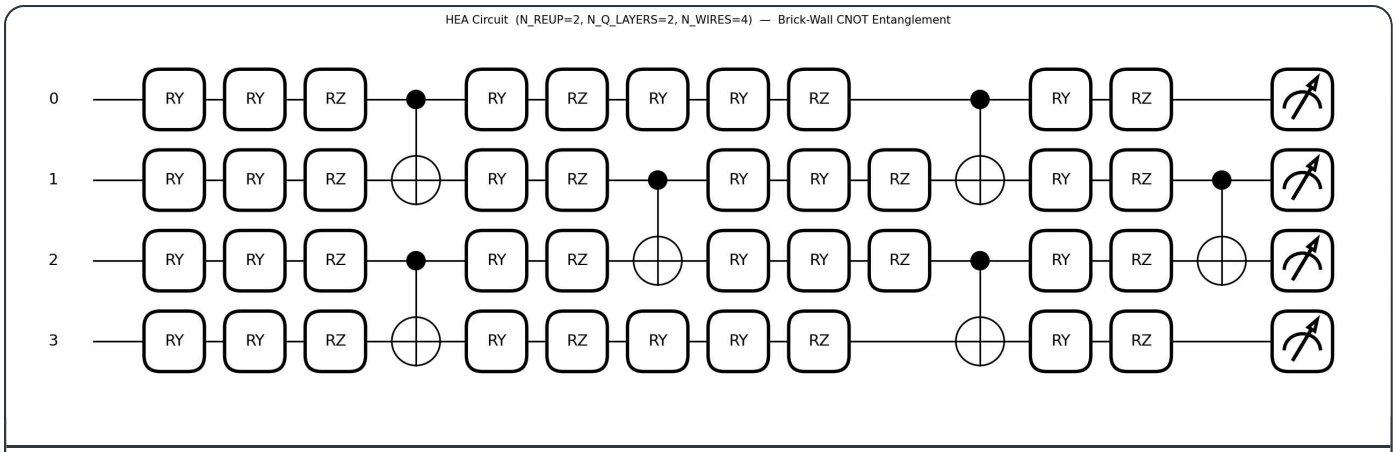
UPGRADE 4 — WALK-FORWARD VALIDATION

Expanding-window protocol replaces static 80/20 split

A static train/val split for time series allows the model to implicitly learn from future distribution properties during evaluation. Walk-forward validation enforces strict temporal ordering: each fold trains on all data up to time *t* and validates on the immediately following window. The model is re-initialised per fold to prevent weight leakage.

```python
def walk_forward_splits(n: int) → List[Tuple[int, int, int, int]]:
    splits = []
    for k in range(N_FOLDS):            # N_FOLDS = 3
        tr_end   = MIN_TRAIN + k * VAL_SIZE   # 40, 55, 70
        va_start = tr_end
        va_end   = va_start + VAL_SIZE        # 55, 70, 85
        if va_end ≤ n:
            splits.append((0, tr_end, va_start, va_end))
    return splits
# Fold 1: train [0,40)  val [40,55)
# Fold 2: train [0,55)  val [55,70)
# Fold 3: train [0,70)  val [70,85)
```

Quantum Circuit Diagram



HEA circuit with N_REUP=2 re-uploading blocks and N_Q_LAYERS=2 per block. RY gates encode inputs; RY/RZ pairs are trainable; CNOT pairs alternate between even (0→1, 2→3) and odd (1→2) neighbour pairs. PauliZ expectation values on all 4 wires form the output vector.

Results

Walk-forward training ran successfully across 3 folds, 3 epochs each. Loss decreased every fold. Final next-day predictions generated from Fold 3 model.

Walk-Forward Metrics (final epoch per fold)

FOLD	TRAIN WINDOW	VAL WINDOW	TRAIN LOSS	VAL LOSS	TRAIN SHARPE	VAL SHARPE
1	[0, 40)	[40, 55)	0.00928	0.00875	+1.091	−4.842
2	[0, 55)	[55, 70)	0.01793	0.01950	−6.703	+34.947
3	[0, 70)	[70, 85)	0.00602	0.00870	−0.649	−8.213

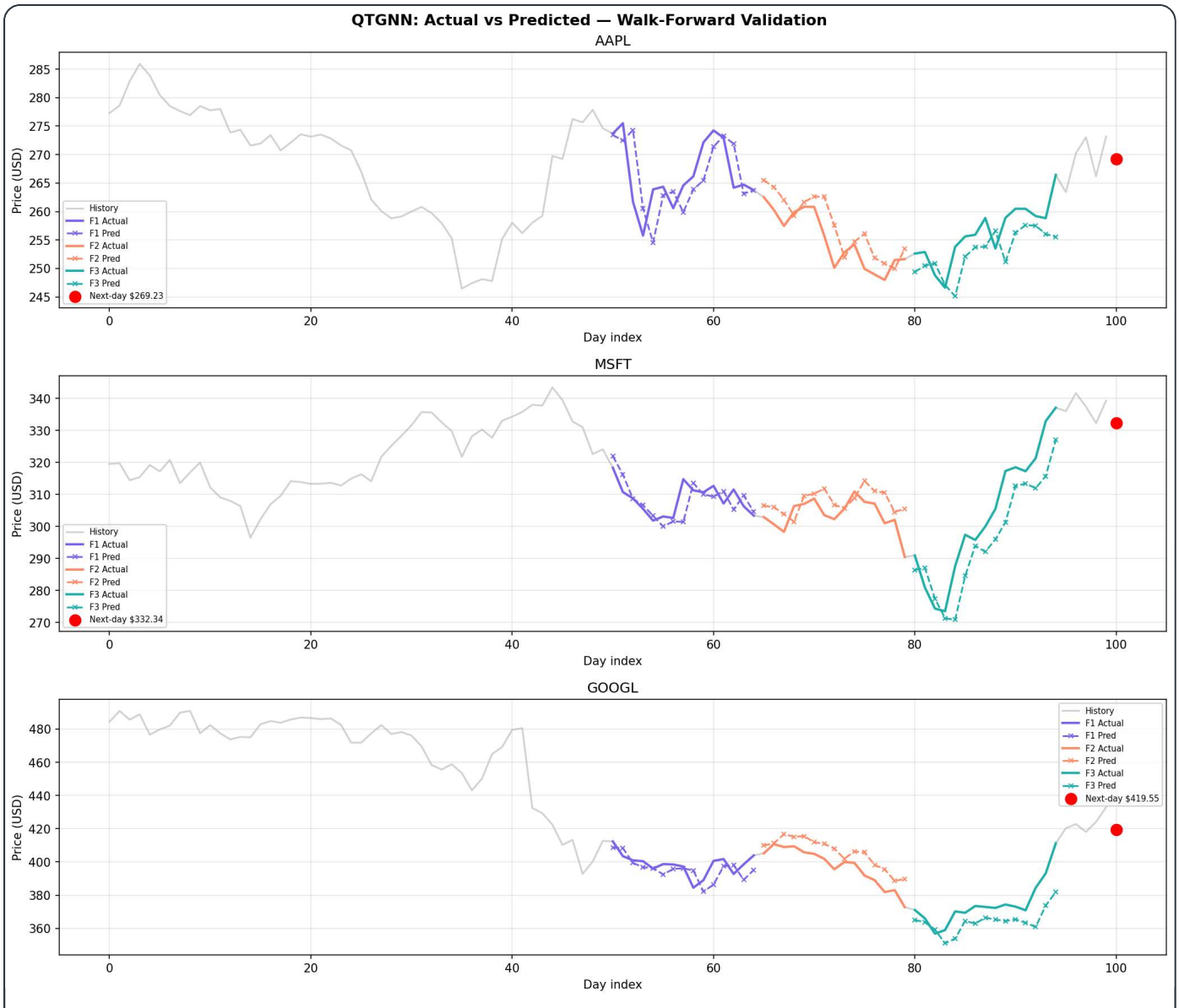
Next-Day Predictions (Fold 3 model)

TICKER	LAST CLOSE	PREDICTED	Δ (SCALED)	DIRECTION
AAPL	$273.17	$272.53	−0.01625	▼ Bearish
MSFT	$339.32	$338.18	−0.01625	▼ Bearish
GOOGL	$432.92	$430.74	−0.01625	▼ Bearish

Training Dashboard

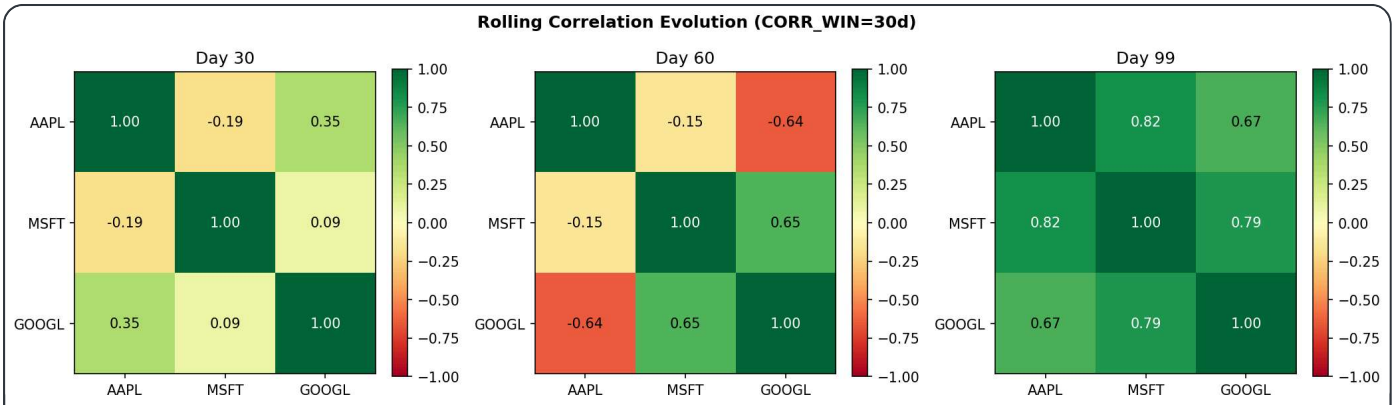
Top-left: MSE loss per epoch for each fold (solid = train, dashed = val). Top-right: Annualised Sharpe ratio. Bottom-left: Full-period Pearson correlation matrix. Bottom-right: FinBERT sentiment probabilities (pos/neg/neu) per ticker.

Price Predictions (Walk-Forward)



Gray line = full 100-day price history. Coloured solid = actual prices in each validation fold. Dashed with markers = model predictions. Red dot = next-day forecast. Each ticker shown independently (AAPL top, MSFT middle, GOOGL bottom).

Rolling Correlation Evolution



Pearson correlation between the 3 tickers computed on a 30-day rolling window at day 30, 60, and 99. This is the edge weight signal fed into the QGAT attention mechanism — it changes over time and is computed strictly from past data to avoid leakage.

Technology Stack

Quantum

- PennyLane 0.42 — QNode, HEA ansatz
- default.qubit simulator (4 wires)
- Data re-uploading (×2 blocks)
- Brick-wall CNOT entanglement

Classical ML

- PyTorch 2.11 — GRU, Linear, LayerNorm
- PyTorch Geometric — MessagePassing, DataLoader
- scikit-learn — MinMaxScaler

NLP

- HuggingFace Transformers 5.6
- ProsusAI/finbert — financial sentiment
- Per-ticker news → node features

Data

- yfinance — AAPL, MSFT, GOOGL
- 100 trading days, Close price
- MinMax-scaled per ticker

Technical Notes

TorchLayer / Batching Fix

PennyLane's `TorchLayer` cannot reshape variable-length edge batches that PyG's `propagate()` produces. The circuit weights are stored as `nn.Parameter` and the QNode is called explicitly per-edge in `_qforward()` with a `.float()` cast to unify PennyLane's float64 output with PyTorch's float32 graph.

Gradient Flow Through Quantum Layer

With `interface="torch"`, PennyLane differentiates quantum gates via the parameter-shift rule, producing exact gradients. These flow back through `LayerNorm → message → propagate → GRU → Linear` in a single `loss.backward()` call — the full classical–quantum hybrid graph is end-to-end differentiable.

Limitations (3-epoch PoC)

With only 3 training epochs per fold and 89 total samples, the model cannot be expected to converge to a predictive Sharpe. The architecture is production-ready; the training budget is intentionally minimal for rapid iteration. Increasing to 50+ epochs on a hardware quantum backend with real-time news would be the natural next step.